

Data Structures and Algorithms

GÉZA HORVÁTH

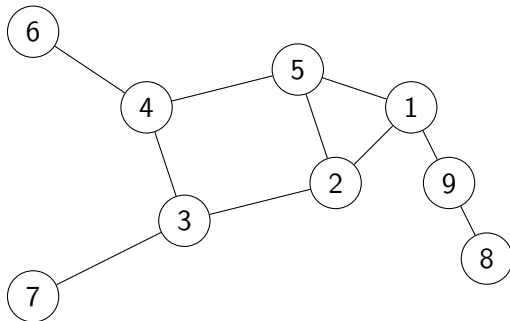
Tenth Lecture

Lecture topics

- 1 Introductory examples of the design and analysis of algorithms.
- 2 Asymptotic behavior of functions, asymptotic notation, further examples of simple algorithms.
- 3 Basic notions concerning data structures: modeling, abstraction. Set, multiset, array, matrix.
- 4 Elementary data structures: stacks, queues, lists.
- 5 Tables, hash tables, hash functions, collisions.
- 6 Trees, binary trees, binary search trees, traversal, searching, insertion, deletion.
- 7 Balanced binary search trees, AVL trees.
- 8 Red-black trees.
- 9 B-trees.
- 10 **Graphs, traversal, shortest path.**
- 11 Parallel algorithms: matrix multiplication, merge sort.
- 12 Decision problems, the complexity classes P and NP.
- 13 Summary.

Undirected graph – abstract data type

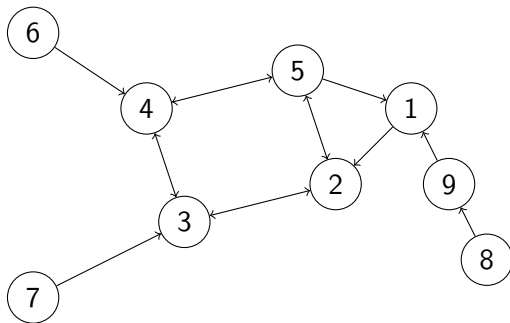
An **undirected graph** data structure consists of a finite set of vertices (also called nodes), together with a set of **unordered pairs** of these vertices. These pairs are known as edges.



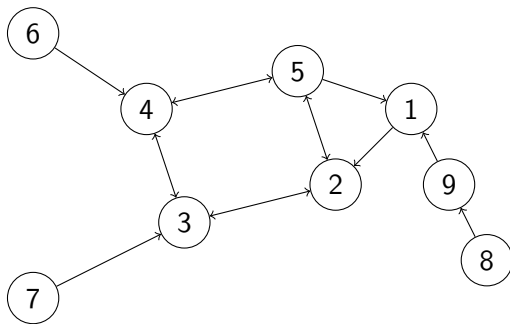
$G = (V, E)$, where $V = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$, and
 $E = \{(1, 2), (1, 5), (1, 9), (2, 3), (2, 5), (3, 4), (3, 7), (4, 6), (8, 9)\}$.

Directed graph – abstract data type

A **directed graph** data structure consists of a finite set of vertices (also called nodes), together with a set of **ordered pairs** of these vertices. These pairs are known as edges, and for a directed graph are also known as arrows.



Directed graph – abstract data type



$G = (V, E)$, where

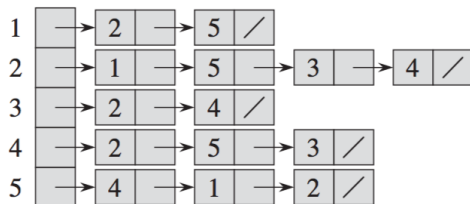
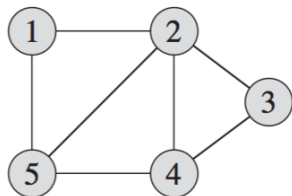
$V = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$, and

$E = \{(1, 2), (2, 3), (2, 5), (3, 2), (3, 4), (4, 3), (4, 5), (5, 1), (5, 2), (5, 4), (6, 4), (7, 3), (8, 9), (9, 1)\}$.

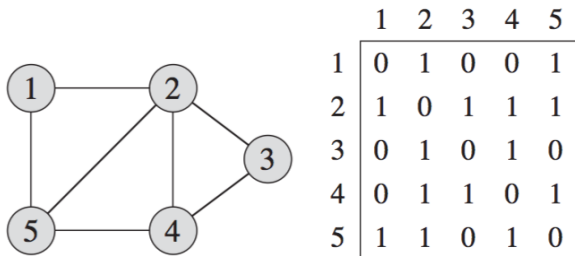
Graph – applications

- Using GPS/Google Maps/Yahoo Maps, to find a route based on shortest route.
- Google, to search for webpages, where pages on the internet are linked to each other by hyperlinks; each page is a vertex and the link between two pages is an edge.
- Connecting with friends on social media, where each user is a vertex, and when users connect they create an edge.
- Computer networks, where computers are vertexes, and connections are edges.
- Graph database is a database that uses graph structures for semantic queries with nodes, edges and properties to represent and store data.

Undirected graph – representation – adjacency-list



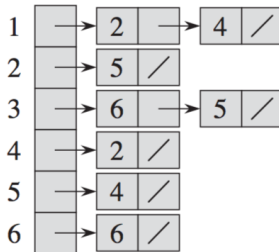
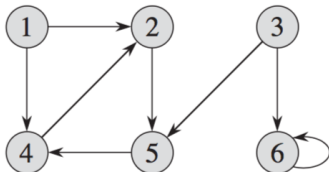
Undirected graph – representation – adjacency-matrix



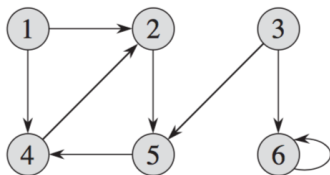
For the **adjacency-matrix representation** of a graph $G = (V, E)$, we assume that the vertices are numbered $1, 2, \dots, |V|$ in some arbitrary manner. Then the adjacency-matrix representation of a graph G consists of a $|V| \times |V|$ matrix $A = (a_{ij})$ such that

$$a_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

Directed graph – representation – adjacency-list



Directed graph – representation – adjacency-matrix



	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

For the **adjacency-matrix representation** of a graph $G = (V, E)$, we assume that the vertices are numbered $1, 2, \dots, |V|$ in some arbitrary manner. Then the adjacency-matrix representation of a graph G consists of a $|V| \times |V|$ matrix $A = (a_{ij})$ such that

$$a_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

Adjacency-list or adjacency-matrix?

① How many elements they have?

- adjacency-list: depends on the number of edges
- adjacency-matrix: $|V|^2$

For **sparse graphs** – those for which $|E|$ is much less than $|V|^2$ – **adjacency-list** is recommended.

For **dense graphs** – those for which $|E|$ is close to $|V|^2$ – **adjacency-matrix** is recommended.

② How long it takes to find an edge or all edges?

- adjacency-matrix: it is fast to lookup and check for presence or absence of a specific edge between any two nodes
- adjacency-list: it is fast to iterate over all edges because you can access any node neighbors directly

Breadth-first search

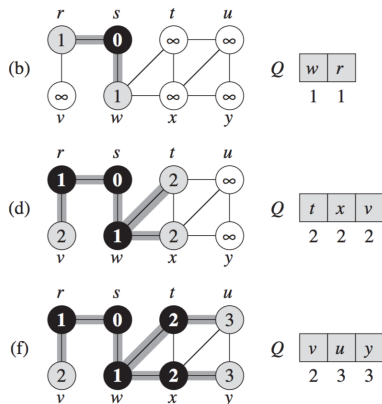
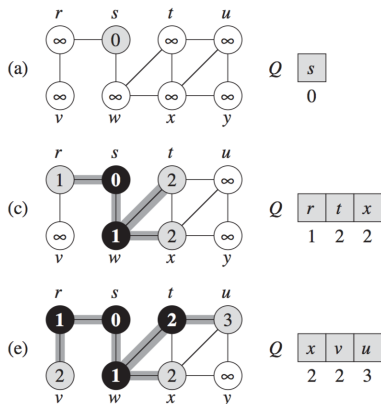
Breadth-first search colors each vertex

- white,
- gray, or
- black.

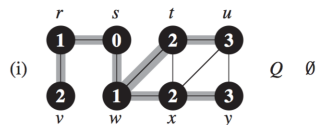
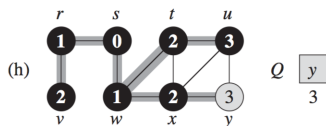
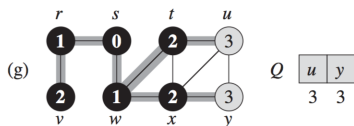
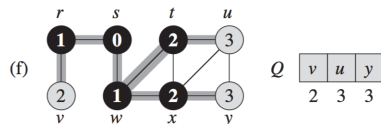
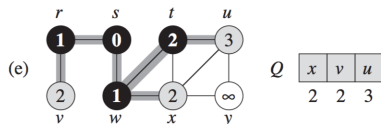
The algorithm also uses a first-in, first-out **queue** Q to manage the set of gray vertices.

The attribute $u.d$ holds the distance from the source s to vertex u .

Breadth-first search



Breadth-first search



Breadth-first search

BFS(G, s)

```

1  for each vertex  $u \in G.V - \{s\}$ 
2       $u.color = \text{WHITE}$ 
3       $u.d = \infty$ 
4       $u.\pi = \text{NIL}$ 
5   $s.color = \text{GRAY}$ 
6   $s.d = 0$ 
7   $s.\pi = \text{NIL}$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11      $u = \text{DEQUEUE}(Q)$ 
12     for each  $v \in G.Adj[u]$ 
13         if  $v.color == \text{WHITE}$ 
14              $v.color = \text{GRAY}$ 
15              $v.d = u.d + 1$ 
16              $v.\pi = u$ 
17             ENQUEUE( $Q, v$ )
18      $u.color = \text{BLACK}$ 
```

The overhead for initialization is $\mathcal{O}(|V|)$, the total time spent in scanning adjacency lists is $\mathcal{O}(|E|)$, and thus the total running time of the BFS procedure is $\mathcal{O}(|V| + |E|)$. Thus, breadth-first search runs in time linear in the size of the adjacency-list representation of G .

Depth-first search

For the depth-first search algorithm we do not use the book!

Depth-first search colors each vertex

- white,
- gray, or
- black.

The algorithm does not use queue, it uses a last-in, last-out **stack** S to manage the set of gray vertices.

Depth-first search

DFS(G, s)

```

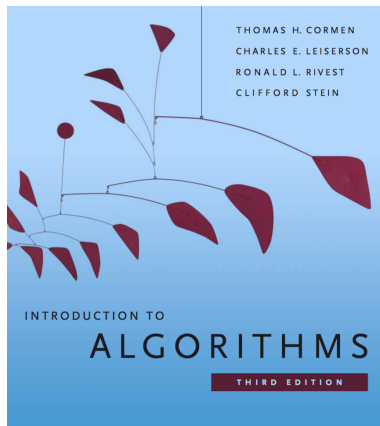
1  for each vertex  $u \in G.V - \{s\}$ 
2       $u.color = \text{WHITE}$ 
3       $u.d = \infty$ 
4       $u.\pi = \text{NIL}$ 
5   $s.color = \text{GRAY}$ 
6   $s.d = 0$ 
7   $s.\pi = \text{NIL}$ 
8  S =  $\emptyset$ 
9  PUSH(S,  $s$ )
10 while S  $\neq \emptyset$ 
11      $u = \text{POP}(\text{S})$ 
12     for each  $v \in G.Adj[u]$ 
13         if  $v.color == \text{WHITE}$ 
14              $v.color = \text{GRAY}$ 
15              $v.d = u.d + 1$ 
16              $v.\pi = u$ 
17             PUSH(S,  $v$ )
18      $u.color = \text{BLACK}$ 

```

The total running time of the DFS procedure is also $\mathcal{O}(|V| + |E|)$.

See the example on the whiteboard!

References



This work was supported by the construction EFOP-3.4.3-16-2016-00021. The project was supported by the European Union, co-financed by the European Social Fund.